

- ▼ 1.使用准备
 - 1.1 python下载与安装
 - ▼ 1.2 编辑器下载
 - 1.2.1 PyChram下载
 - 1.2.2 VScode下载、安装与配置
 - ▼ 1.3第三方库的下载与安装
 - 1.3.1使用终端/terminal安装第三方库
 - 1.3.2使用PyCharm安装第三方库
 - 1.4打开程序
- ▼ 2.开始使用
 - ▼ 2.1训练模型
 - 2.1.1检查是否有缺失值
 - 2.1.2修改参数
 - 2.1.3 结果查看
 - 2.1.4 堆叠法
 - ▼ 2.2.保存模型
 - 2.2.1 修改参数
 - 2.2.2 结果查看
 - ▼ 2.3使用模型预测
 - 2.3.1修改文件路径
 - 2.3.2修改函数
 - 2.3.3结果查看
- ▼ 3.代码结构
 - 3.1 main.py
 - 3.2 loop_models.py
 - 3.3 MLP_Classiffication_Sklearn.py
 - 3.4 predict.py

1.使用准备

1.1 python下载与安装

有关python下载与安装请见[Python的下载安装教程](#)

1.2 编辑器下载

编辑器可用于修改与运行代码，这里提供了PyCharm与VScode的下载与安装两种方法

1.2.1 PyCharm下载

有关PyCharm下载与安装请见[PyCharm安装教程](#)

1.2.2 VScode下载、安装与配置

有关VScode下载与安装请见[用VScode配置Python开发环境](#)

1.3 第三方库的下载与安装

在本程序中使用了numpy、pandas、sklearn库，在运行程序前请确保您已经下载并安装了所有需要的第三方库，这里提供了从终端中安装与从PyCharm中安装两种方法

1.3.1 使用终端/terminal安装第三方库

1. 使用 `win+R` 打开'运行'对话框。
2. 在对话框中输入`·cmd·`，点击确定
3. 分行输入以下代码

注：一次仅输入一行，运行完成后再按顺序输入下一行。

```
pip install numpy
pip install pandas
pip install scipy
pip install matplotlib
pip install scikit_learn
pip install openpyxl
pip install xlswriter
pip install joblib
```

如果发生网络错误，试试以下换源代码

```
pip install [第三方库名] --trusted-host pypi.mirrors.ustc.edu.cn/simple # 中科大源
pip install [第三方库名] --trusted-host pypi.tuna.tsinghua.edu.cn/simple # 清华源
```

1.3.2使用PyCharm安装第三方库

有关使用PyCharm下载安装第三方库请见[安装第三方库的四种方法](#)

1.4打开程序

打开编辑器，点击左上角的File，再点击Open，选择文件夹所在的位置，点击确定。

2.开始使用

请注意:请将数据以.xlsx格式保存，包含n个特征列与1个标签列。以下操作均使用 data 文件夹下的 subdimension-data.xlsx 文件为例，该文件共包含39列，其中第1-38列为特征，第39列为标签。每一次运行结束后，请及时将 # 恢复，以免影响下一步操作。

2.1训练模型

以下涉及到不同的算法部分操作均以 programs 文件夹下的 MLP_Classiffication_Sklearn.py 为例

2.1.1检查是否有缺失值

在 programs 文件夹下的 if_miss.py 中进行

1. 修改文件路径

将 file = 'E:\\Classiffication_Models\\real\\subdimension-data.xlsx' 修改为当前 subdimension-data.xlsx 文件所在的绝对路径

2. 运行程序 if_miss.py

3. 如果每一行最后数值均为0，则无缺失值，进入下一节。若不为零，则存在缺失值，请检查文件。

2.1.2修改参数

在 programs 文件夹下的 main.py 中进行

1. 将 file = 'E:\\Classiffication_Models\\real\\subdimension-data.xlsx' 修改为当前 subdimension-data.xlsx 文件所在的绝对路径

2. 打开 main.py，找到：

```

test_sizes = [0.2,0.3]
# MLP parameters:
hidden_layer_sizeses = [(70,35),(60,30),(50,25),(40,20),(30,15),(20,10)]# list of tups like (10,10)
activations = ['tanh', 'relu', 'logistic', 'identity']# list of strs among {'tanh', 'relu', 'logistic', 'identity'}
solvers = ['lbfgs', 'adam', 'sgd']# list of strs among {'lbfgs', 'adam', 'sgd'}
alphas = [0.01,0.1,1]# list of floats in the range [0.0, inf)
max_iters = [500,700,1000,1500]# list of ints in the range [1, inf)

```

通过修改列表/list（中括号）中的字符串、元组/tuple（小括号）、整数或小数来得到不同的参数组合以得到不同的结果。

3. 修改 main()

找到

```

def main():
    """
    loop the models
    ## MLP:
    loop_mlp(file,hidden_layer_sizeses,activations,solvers,alphas,max_iters)
    ## SVM:
    loop_svm(file,test_sizes,kernels,Cs,gammas,decision_function_shapes,shrinkings,probabilities)
    ## RandomForest:
    loop_rf(file,test_sizes,n_estimatorses, criteria,bootstraps, oob_scores)
    ## KNN:
    loop_knn(file,test_sizes,n_neighborses,weightses,algorithms)

    ## Stack:
    loop_stack(file,test_sizes,estimatorses,final_estimators)

    """
    #loop_mlp(file,test_sizes,hidden_layer_sizeses,activations,solvers,alphas,max_iters)
    #loop_svm(file,test_sizes,kernels,Cs,gammas,decision_function_shapes,shrinkings,probabilities)
    #loop_rf(file,test_sizes,n_estimatorses, criteria,bootstraps, oob_scores)
    #loop_knn(file,test_sizes,n_neighborses,weightses,algorithms)
    #loop_stack(file,test_sizes,estimatorses,final_estimators)

```

这里可以控制运行那些算法，将loop_mlp()前的 # 删除。

4. 运行程序 main.py

2.1.3 结果查看

由于此处我们仅运行了MLP,因此只会出现一个 mlp_result.xlsx 的文件。包含各个参数组合及运行结果。

2.1.4 堆叠法

将四种算法全部运行完成之后，可以尝试使用堆叠法来得到更好的精度与泛化能力，选择四种算法中最好的几种参数组合，在 `main.py` 中修改 `estimatorses` 与 `final_estimators` 内的参数，其中 `estimatorses` 为基学习器的参数，`final_estimators` 为元学习器的参数。

注意：对于 `estimatorses` 一个列表/list为一个参数，注意修改时的格式

2.2.保存模型

开始保存模型之前，请您确保您已经确认数据文件无缺失值，并且已经将 `file` 修改为您当前的数据保存路径

2.2.1 修改参数

在 `programs` 文件夹下的 `main.py` 中进行，建议执行此步骤之前，通过2.1得到最佳参数组合。

1. 打开 `main.py` ,将 `file = 'E:\\\\Classification_Models\\\\real\\\\subdimension-data.xlsx` 修改为当前 `subdimension-data.xlsx` 文件所在的绝对路径
2. 打开 `main.py` , 找到:

```
# MLP parameters:
hidden_layer_sizeses = [(70,35),(60,30),(50,25),(40,20),(30,15),(20,10)]# list of tuples like (10,10)
activations = ['tanh', 'relu', 'logistic', 'identity']# list of strs among {'tanh', 'relu', 'logistic', 'identity'}
solvers = ['lbfgs', 'adam', 'sgd']# list of strs among {'lbfgs', 'adam', 'sgd'}
alphas = [0.01,0.1,1]# list of floats in the range [0.0, inf)
max_iters = [500,700,1000,1500]# list of ints in the range [1, inf)
```

通过修改列表/list（中括号）中的字符串、元组/tuple（小括号），将其修改为2.1.3中得到的最佳的参数组合。若在一个列表中保留多个，程序会自动执行列表中的第一个参数。注意，保存模型时是使用全部数据训练，因此无需修改 `test_sizes`

3. 修改 `main()`

找到

```
# save the models:
#save_mlp(file,hidden_layer_sizeses[0],activations[0],solvers[0],alphas[0],max_iters[0])
#save_svm(file,kernels[0],Cs[0],gammas[0],decision_function_shapes[0],shrunkings[0],probability
#save_rf(file,n_estimatorses[0], criteriaes[0],bootstraps[0], oob_scores[0])
#save_knn(file,n_neighborses[0],weightses[0],algorithms[0])
#save_stack(file,estimatorses[0],final_estimators[0])
```

将 `#save_mlp(file,hidden_layer_sizeses[0],activations[0],solvers[0],alphas[0],max_iters[0])` 前的 `#` 删除

4. 运行程序 `main.py`

2.2.2 结果查看

由于此处我们仅运行了 `save_mlp()` ,因此只会出现一个 `mlp_model.joblib` 的文件。请将其保存在 `saved_models` 文件夹下。

2.3使用模型预测

这里使用 `data` 文件夹下的 `test.xlsx` 文件作为例子, 请注意: `test.xlsx` 文件的格式应当和 `data` 文件夹下的 `subdimension-data.xlsx` 保持一致, 除了没有标签列。

2.3.1修改文件路径

1. 打开 `main.py`

找到

```
predict_file = 'E:\\Classiffication_Models\\Classiffication_Models-v1.2.1\\data\\test.xlsx'

mlp_model = 'E:\\Classiffication_Models\\Classiffication_Models-v1.2.1\\saved_models\\mlp_model.
svm_model = 'E:\\Classiffication_Models\\Classiffication_Models-v1.2.1\\saved_models\\svm_model.
rf_model = 'E:\\Classiffication_Models\\Classiffication_Models-v1.2.1\\saved_models\\rf_model.jc
knn_model = 'E:\\Classiffication_Models\\Classiffication_Models-v1.2.1\\saved_models\\knn_model.
stack_model = 'E:\\Classiffication_Models\\Classiffication_Models-v1.2.1\\saved_models\\stack_mc
```

将 `predict_file` 与 `mlp_model` 文件分别更改为 `test.xlsx` 与 `mlp_model.joblib` 的绝对路径

2.3.2修改函数

1. 打开 `main.py`

2. 修改 `main()`

找到

```
# predict
#mlp_predict(predict_file,mlp_model)
#svm_predict(predict_file,svm_model)
#rf_predict(predict_file,rf_model)
#knn_predict(predict_file,knn_model)
#stack_predict(predict_file,stack_model)
```

将 `#mlp_predict(predict_file,mlp_model)` 前的 `#` 删去

3. 运行 `main()`

2.3.3结果查看

运行结果将会保存在 `data` 文件夹下的 `test.xlsx` 文件的 `mlp_pred` 工作表中

3.代码结构

以MLP方法为例

3.1 main.py

`main()` 方法, 调用 `loop_models.py` 中的 `loop_mlp()` 方法, 给 `loop_mlp()` 传入参数 `file`, `test_sizes`, `hidden_layer_sizes`, `activations`, `solvers`, `alphas`, `max_iters`, 调用 `MLP_Classification_Sklearn.py` 中的 `save_mlp()` 方法, 传入参数 `file`, `test_size`, `hidden_layer_sizes`, `activation`, `solver`, `alpha`, `max_iter`, 调用 `predict.py` 中的 `mlp_predict()` 方法, 传入参数 `predict_file`, `mlp_model`

3.2 loop_models.py

`loop_mlp()` 方法得到参数 `file`, `test_sizes`, `hidden_layer_sizes`, `activations`, `solvers`, `alphas`, `max_iters`, 其中 `file` 不参与循环, 将其余参数嵌套循环, 形成多种参数组合, 每一个组合调用一次 `MLP_Classification_Sklearn.py` 中的 `mlp()` 方法, 给 `mlp()` 方法传入参数 `file`, `test_size`, `hidden_layer_sizes`, `activation`, `solver`, `alpha`, `max_iter`, 得到 `mlp()` 返回值 `mlp_result`, 将 `mlp_result` 存入到列表 `mlp_results` 中。创建了一个Excel写入器 `mlp_writer`, 将结果保存在 `mlp_result.xlsx` 中的 `mlp sheet`中, 遍历 `mlp_results`, `mlp_writer` 将 `mlp_results` 中的 `mlp_result` 保存入 `mlp_result.xlsx` 中。

3.3 MLP_Classification_Sklearn.py

`mlp()` 方法得到 `file`, `test_size`, `hidden_layer_sizes`, `activation`, `solver`, `alpha`, `max_iter` 参数, 通过 `file` 读取文件, 预处理数据, 通过 `test_size` 切分训练集与测试集, 通过 `hidden_layer_sizes`, `activation`, `solver`, `alpha`, `max_iter` 建立模型与训练模型, 返回包含参数组合、测试结果、哥特征的贡献程度的 `mlp_result`

`save_mlp()` 方法得到 `file, test_size, hidden_layer_sizes, activation, solver, alpha, max_iter` 参数, 训练并保存mlp模型。

3.4 predict.py

`mlp_predict()` 方法得到参数 `predict_file, mlp_model` , 根据模型对数据进行预测, 并将结果保存。